

```

// Storage layer – authenticated, offline-
capable, conflict-safe.
//
// The browser no longer touches the
database table directly. Every request
// goes through a database function that
checks a session token first:
//
//   staff_login      username + password
-> session token (password bcrypt-checked)
//   state_get        token
-> the whole school
//   state_save       token + data
-> conditional write, refuses stale saves
//   student_record  admission no + PIN
-> ONLY that child's records
//
// So even with the site link and the
public key, an outsider gets nothing:
// the table itself rejects direct reads
and writes.
//
// Vercel → Settings → Environment
Variables:
//   VITE_SUPABASE_URL      =
https://<project-ref>.supabase.co
//   VITE_SUPABASE_ANON_KEY = <publishable
/ anon key>

const URL_BASE =

```

```

import.meta.env.VITE_SUPABASE_URL;
const ANON_KEY =
import.meta.env.VITE_SUPABASE_ANON_KEY;

export const isShared = Boolean(URL_BASE &&
ANON_KEY);

const MIRROR_KEY = "school_mirror_v2";
const BASE_KEY = "school_base_v2";
const PENDING_KEY = "school_pending_v2";
const SCHOOL_KEY = "school_code_v1";
const TOKEN_KEY = "school_token_v1";
const WHO_KEY = "school_who_v1";

const readJSON = (k) => { try { const r =
localStorage.getItem(k); return r ?
JSON.parse(r) : null; } catch (e) { return
null; } };
const writeJSON = (k, v) => { try {
localStorage.setItem(k, JSON.stringify(v));
} catch (e) {} };
const del = (k) => { try {
localStorage.removeItem(k); } catch (e) {}
};

export const hasPendingChanges = () => {
try { return
localStorage.getItem(PENDING_KEY) === "1";
} catch (e) { return false; } };
const setPending = (v) => { try { v ?
localStorage.setItem(PENDING_KEY, "1") :
del(PENDING_KEY); } catch (e) {} };
export const hasPendingSave = () => { try {
return localStorage.getItem(PENDING_KEY)

```

```

=== "1"; } catch (e) { return false; } };
```



```

// Retrying automatically when the signal
returns.
//
// Work is never lost – saveRoster writes
the device copy before it ever tries
// the network. What was missing is this:
without it, a mark entered offline
// sat on the phone until someone happened
to open the app again and press
// Save a second time. A teacher who
entered a whole class's marks with no
// signal and then went home has no reason
to reopen the portal that evening.
//
// The browser's own "online" event fires
the moment the phone reconnects –
// no polling, no cost while offline.
let retrying = false;
let onSynced = null;
export const onSyncStateChange = (cb) => {
onSynced = cb; };

async function retryPendingSave() {
  if (retrying || !hasPendingSave() ||
!isShared) return;
  retrying = true;
  try {
    const local = readJSON(MIRROR_KEY);
    if (local) {
      await saveRoster(local);
      onSynced?.("synced");
    }
  }
}

```

```

    } catch (e) {
        // still cannot reach the database –
        stay pending, try again next time
        // the browser tells us we are back
        online
    } finally {
        retrying = false;
    }
}

if (typeof window !== "undefined") {
    window.addEventListener("online",
        retryPendingSave);
    // a phone can regain a weak signal
    without firing "online" reliably, so a
    // slow background check catches what the
    event misses – cheap, because it
    // does nothing at all unless there is
    genuinely a save waiting
    setInterval(() => { if (hasPendingSave())
        retryPendingSave(); }, 45000);
}

export const isOffline = () => typeof
navigator !== "undefined" &&
navigator.onLine === false;

export const getToken = () => { try {
    return localStorage.getItem(TOKEN_KEY); }
    catch (e) { return null; } };
export const getWho    = () =>
readJSON(WHO_KEY);
const setSession = (token, who) => {
    if (token) { try {
        localStorage.setItem(TOKEN_KEY, token); }

```

```

catch (e) {} writeJSON(WHO_KEY, who); }
else { del(TOKEN_KEY); del(WHO_KEY); }
};

let lastSeenUpdatedAt = null;

async function rpc(fn, args) {
  const res = await
  fetch(`${URL_BASE}/rest/v1/rpc/${fn}`, {
    method: "POST",
    headers: { apikey: ANON_KEY,
  Authorization: `Bearer ${ANON_KEY}`,
  "Content-Type": "application/json" },
    body: JSON.stringify(args || {}),
  });
  if (!res.ok) {
    const txt = await res.text().catch(()
=> "");
    let detail = txt;
    try { const j = JSON.parse(txt); detail
= j.message || j.hint || j.details || txt;
  } catch (e) {}
    throw new Error(`${fn} ${res.status}:
${String(detail).slice(0, 160)}`);
  }

  // Functions that return nothing send an
empty body (HTTP 204). Parsing that
  // as JSON throws "Unexpected end of JSON
input", which looked like a failure
  // even though the work had already
succeeded.
  if (res.status === 204) return null;
  const text = await res.text();

```

```

    if (!text) return null;
    try { return JSON.parse(text); } catch
(e) { return null; }
}

// ----- authentication -----
// The school is named once, here, and
checked against the password. After this
// the token carries it – nothing else ever
accepts a school as an argument, so
// there is no request that can reach
across to another school.
export async function staffLogin(school,
username, password) {
    if (!isShared) throw new Error("No
database configured");
    const rows = await rpc("staff_login", {
        p_school: school, p_username: username,
p_password: password });
    const row = Array.isArray(rows) ? rows[0]
: rows;
    if (!row || !row.token) return
null; // wrong school,
username or password
    const who = {
        role: row.role, name: row.name,
teacherId: row.teacher_id, username,
        mustChange: !!row.must_change,
        schoolCode: row.school_code,
schoolName: row.school_name,
        schoolLocation: row.school_location,
isOwner: !!row.is_owner,
    };
    setSession(row.token, who);
}

```

```

    try { localStorage.setItem(SCHOOL_KEY,
school); } catch (e) {}
    return who;
}

// The schools this portal serves, for the
sign-in screen. Names and codes only.
export const listSchools = () =>
rpc("schools_list", {});

// The school last used on this device, so
a teacher is not asked every morning.
export const lastSchool = () => { try {
return localStorage.getItem(SCHOOL_KEY) ||
""; } catch (e) { return ""; } };

export async function staffLogout() {
    const t = getToken();
    setSession(null, null);
    del(MIRROR_KEY); del(BASE_KEY);
del(PENDING_KEY); // don't leave school
data on a shared phone
    if (t && isShared) { try { await
rpc("staff_logout", { p_token: t }); }
catch (e) {} }
}

// Restores a session after a reload; null
means the token expired.
export async function restoreSession() {
    const t = getToken();
    if (!t || !isShared) return null;
    try {
        const rows = await rpc("staff_me", {

```

```

p_token: t });
    const row = Array.isArray(rows) ?
rows[0] : rows;
    if (!row) { setSession(null, null);
return null; }
    const who = { role: row.role, name:
row.name, teacherId: row.teacher_id,
                username: row.username,
mustChange: !!row.must_change,
                schoolCode:
row.school_code, schoolName:
row.school_name,
                schoolLocation:
row.school_location, isOwner:
!!row.is_owner };
    writeJSON(WHO_KEY, who);
    return who;
  } catch (e) {
    return getWho();      // offline:
trust the cached identity until reconnect
  }
}

export const changeMyPassword = async
(oldPw, newPw) => {
  // kept for the older screen that calls
it; both routes now go through the
  // one function that enforces the
password rules
  await rpc("staff_change_own_password", {
p_token: getToken(), p_current: oldPw,
p_new: newPw });
  return true;
};

```



```

// ----- staff accounts (admin) -----
----
export const staffList = () =>
rpc("staff_list", { p_token: getToken() });
export const staffUpsert = (username, name,
role, password, teacherId) =>
  rpc("staff_upsert", { p_token:
getToken(), p_username: username, p_name:
name, p_role: role, p_password: password ||
null, p_teacher_id: teacherId || null });
export const staffDeactivate = (username)
=> rpc("staff_deactivate", { p_token:
getToken(), p_username: username });

// Changes ONLY the password. Using
staffUpsert for this previously overwrote
// the person's role, which silently
demoted administrators.
export const staffResetPassword =
(username, newPassword) =>
  rpc("staff_reset_password", { p_token:
getToken(), p_username: username, p_new:
newPassword });

// ----- parents -----
export async function
parentLookup(admissionNo, pin) {
  if (!isShared) return null;
  return rpc("student_record", { p_adm:
admissionNo, p_pin: pin });
}

// ----- three-way merge (unchanged:

```

```

protects simultaneous saves) -----
function mergeValue(base, mine, yours) {
  if (mine === yours) return mine;
  if (JSON.stringify(mine) ===
JSON.stringify(base)) return yours;
  if (JSON.stringify(yours) ===
JSON.stringify(base)) return mine;
  if (Array.isArray(mine) &&
Array.isArray(yours)) return
mergeArrays(base, mine, yours);
  if (mine && yours && typeof mine ===
"object" && typeof yours === "object") {
    const out = { ...yours };
    new Set([...Object.keys(mine),
...Object.keys(yours)]).forEach((k) => {
      const b = base && typeof base ===
"object" ? base[k] : undefined;
      if (!(k in mine)) { if
(JSON.stringify(yours[k]) ===
JSON.stringify(b)) delete out[k]; }
      else out[k] = mergeValue(b, mine[k],
yours[k]);
    });
    return out;
  }
  return mine;
}

function mergeArrays(base, mine, yours) {
  const hasIds = (a) => Array.isArray(a) &&
a.every((x) => x && typeof x === "object"
&& "id" in x);
  if (!hasIds(mine) || !hasIds(yours))
return mine;
  const baseArr = Array.isArray(base) ?

```

```

base : [];
  const byId = (arr) =>
Object.fromEntries(arr.map((x) => [x.id,
x]));
  const B = byId(baseArr), M = byId(mine);
  const out = []; const seen = new Set();
  yours.forEach((y) => {
    seen.add(y.id);
    if (y.id in M)
out.push(mergeValue(B[y.id], M[y.id], y));
    else if (!(y.id in B)) out.push(y);
  });
  mine.forEach((m) => { if (!seen.has(m.id)
&& !(m.id in B)) out.push(m); });
  return out;
}

// ----- reading / saving the school -
-----
export async function loadRoster() {
  if (!isShared) return
readJSON(MIRROR_KEY);
  if (hasPendingChanges()) { const local =
readJSON(MIRROR_KEY); if (local) return
local; }

  try {
    const rows = await rpc("state_get", {
p_token: getToken() });
    const row = Array.isArray(rows) ?
rows[0] : rows;
    const data = row?.data ?? null;
    lastSeenUpdatedAt = row?.updated_at ??
null;

```

```

    if (data) { writeJSON(MIRROR_KEY,
data); writeJSON(BASE_KEY, data); }
    return data;
  } catch (e) {
    const local = readJSON(MIRROR_KEY);
    if (local) return local;          //
offline
    throw e;
  }
}

export async function saveRoster(data) {
  writeJSON(MIRROR_KEY, data);      //
device copy first – never lose work
  if (!isShared) return;

  try {
    let rows = await rpc("state_save", {
p_token: getToken(), p_data: data,
p_expected: lastSeenUpdatedAt });
    let row = Array.isArray(rows) ? rows[0]
: rows;

    if (row && row.ok === false) {
      // Someone else saved first – merge
their version with ours and retry.
      const base = readJSON(BASE_KEY) ||
{};
      const merged = mergeValue(base, data,
row.data || {});
      lastSeenUpdatedAt = row.updated_at;
      rows = await rpc("state_save", {
p_token: getToken(), p_data: merged,
p_expected: lastSeenUpdatedAt });

```

```

        row = Array.isArray(rows) ? rows[0] :
rows;
        if (!row?.ok) throw new Error("Busy –
another device is saving. Retrying...");
        lastSeenUpdatedAt = row.updated_at;
        writeJSON(MIRROR_KEY, merged);
writeJSON(BASE_KEY, merged);
        setPending(false);
        return merged; //
caller adopts the merged result
    }

    lastSeenUpdatedAt = row?.updated_at ??
lastSeenUpdatedAt;
    writeJSON(BASE_KEY, data);
    setPending(false);
} catch (e) {
    setPending(true);
    throw e;
}
}

// ----- password reset -----
export async function
requestReset(username) {
    const res = await fetch("/api/forgot-
password", {
        method: "POST",
        headers: { "Content-Type":
"application/json" },
        body: JSON.stringify({ username }),
    });
    if (!res.ok) {
        const j = await res.json().catch(() =>

```

```

({}));
    throw new Error(j.error || `Reset
request failed (${res.status})`);
  }
  return true;
}

export async function
confirmReset(username, code, newPassword) {
  const r = await rpc("reset_confirm", {
    p_username: username, p_code: code, p_new:
    newPassword });
  return r === true;
}

export const staffSetEmail = (username,
email) =>
  rpc("staff_set_email", { p_token:
getToken(), p_username: username, p_email:
email });

export const staffSetContact = (username,
email, phone) =>
  rpc("staff_set_contact", { p_token:
getToken(), p_username: username, p_email:
email || null, p_phone: phone || null });

// Public: the login screen needs the
school name before anyone signs in.
export async function schoolInfo() {
  if (!isShared) return null;
  try {
    const rows = await rpc("school_info",
{});

```

```

    const r = Array.isArray(rows) ? rows[0]
    : rows;
    return r ? { name: r.name, location:
r.location, motto: r.motto } : null;
  } catch (e) {
    return null;
  }
}

// ----- student photos -----
// Stored in their own table so the roster
// stays small and quick to save.
export const photoSet = (studentId,
dataUrl) => rpc("photo_set", { p_token:
getToken(), p_student: studentId, p_data:
dataUrl });
export const photoDelete =
(studentId) => rpc("photo_delete",
{ p_token: getToken(), p_student: studentId
});
export const photosWhich =
() => rpc("photos_which",
{ p_token: getToken() });

export async function photosGet(studentIds)
{
  if (!isShared || !studentIds?.length)
  return {};
  const rows = await rpc("photos_get", {
p_token: getToken(), p_students: studentIds
});
  const out = {};
  (rows || []).forEach((r) => {
out[r.student_id] = r.data; });

```

```

    return out;
}

// ----- system health & snapshots ---
-----
export const healthCheck = ()
=> rpc("health_check", { p_token:
getToken() });
export const backupsList = ()
=> rpc("backups_list", { p_token:
getToken() });
export const backupNow = (reason)
=> rpc("backup_now", { p_token:
getToken(), p_reason: reason || "manual"
});
export const backupRestore = (id)
=> rpc("backup_restore", { p_token:
getToken(), p_id: id });

// ----- M-Pesa reconciliation -----
---
// Codes are claimed in their own table
with a uniqueness rule, so the same
// confirmation code can never be recorded
twice.
export const mpesaClaim = (code, studentId,
amount, date, sender) =>
    rpc("mpesa_claim", { p_token: getToken(),
p_code: code, p_student: studentId,
                        p_amount: amount,
p_date: date, p_sender: sender || null });
export const mpesaLookup = (code) =>
rpc("mpesa_lookup", { p_token: getToken(),
p_code: code });

```



```

export const mpesaRecent = (days) =>
  rpc("mpesa_recent", { p_token: getToken(),
    p_days: days || 30 });
export const mpesaRelease = (code) =>
  rpc("mpesa_release", { p_token: getToken(),
    p_code: code });

// ----- geofence -----
// Some actions should only happen at
// school. Position can be faked, so this is
// a deterrent against convenience rather
// than a lock against determination –
// every check is written to an audit trail
// either way.
export const geofenceGet = () =>
  rpc("geofence_get", { p_token: getToken()
  });
export const geofenceSet = (v) =>
  rpc("geofence_set", { p_token: getToken(),
    p_value: v });
export const locationRecent = (days) =>
  rpc("location_recent", { p_token:
    getToken(), p_days: days || 14 });
export const locationRecord = (action, pos,
  distance, inside, note) =>
  rpc("location_record", {
    p_token: getToken(), p_action: action,
    p_lat: pos?.lat ?? null, p_lng:
pos?.lng ?? null,
    p_accuracy: pos?.accuracy ?? null,
  p_distance: distance ?? null,
    p_inside: inside, p_note: note || null,
  });

```

```

// Metres between two points on the earth's
surface.
export function metresBetween(a, b) {
  const R = 6371000, rad = (d) => (d *
Math.PI) / 180;
  const dLat = rad(b.lat - a.lat), dLng =
rad(b.lng - a.lng);
  const x = Math.sin(dLat / 2) ** 2 +
    Math.cos(rad(a.lat)) *
Math.cos(rad(b.lat)) * Math.sin(dLng / 2)
** 2;
  return Math.round(2 * R *
Math.asin(Math.sqrt(x)));
}

// Asks the device where it is. Rejects
with a plain-language reason.
//
// maximumAge is 0 deliberately. A cached
fix would let someone pass a check
// using a reading taken minutes earlier at
the school gate, which defeats the
// point of asking. Every check costs a
fresh fix.
export function currentPosition({ timeout =
15000 } = {}) {
  return new Promise((resolve, reject) => {
    if (!navigator.geolocation) return
reject(new Error("This device cannot report
its location."));

    navigator.geolocation.getCurrentPosition(
      (p) => resolve({ lat:
p.coords.latitude, lng: p.coords.longitude,

```

```

accuracy: Math.round(p.coords.accuracy) }},
    (e) => reject(new Error(
        e.code === 1 ? "Location permission
was refused. Allow it in your browser
settings to continue."
        : e.code === 2 ? "Your position
could not be found. Step outside or nearer
a window and try again."
        : "Finding your position took too
long. Try again.")),
    { enableHighAccuracy: true, timeout,
maximumAge: 0 }
    );
});
}

// ----- leave applications -----
export const leaveApply = (kind, starts,
ends, reason, cover) =>
    rpc("leave_apply", { p_token: getToken(),
p_kind: kind, p_starts: starts,
                        p_ends: ends,
p_reason: reason, p_cover: cover });
export const leavelist = (all) =>
    rpc("leave_list", { p_token: getToken(),
p_all: !!all });
export const leaveDecide = (id, status,
note) =>
    rpc("leave_decide", { p_token:
getToken(), p_id: id, p_status: status,
p_note: note });
export const leaveCancel = (id) =>
    rpc("leave_cancel", { p_token: getToken(),
p_id: id });

```

```

export const leaveToday = () =>
rpc("leave_today", { p_token: getToken()
});

// ----- expenditure -----
// Fees collected are only half the
account; where the money went is the half
// people ask about.
export const expenseCategories = () =>
rpc("expense_categories_list", {});
export const expenseAdd = (e) =>
rpc("expense_add", {
  p_token: getToken(), p_date: e.date,
  p_category: e.category, p_description:
e.description,
  p_amount: e.amount, p_paid_to: e.paidTo
|| null, p_method: e.method || "cash",
  p_reference: e.reference || null,
  p_approved_by: e.approvedBy || null,
  p_note: e.note || null,
});
export const expenseList = (from, to) =>
rpc("expense_list", { p_token:
getToken(), p_from: from || null, p_to: to
|| null });
export const expenseSummary = (from, to) =>
rpc("expense_summary", { p_token:
getToken(), p_from: from || null, p_to: to
|| null });
export const expenseDelete = (id, reason)
=> rpc("expense_delete", { p_token:
getToken(), p_id: id, p_reason: reason });

// ----- holiday work -----

```

```

export const assignmentSave = (a) =>
rpc("assignment_save", {
  p_token: getToken(), p_id: a.id || null,
  p_class: a.classId, p_subject: a.subject,
  p_title: a.title, p_instructions:
a.instructions || null, p_body: a.body,
  p_due: a.due || null, p_published:
a.published !== false,
});
export const assignmentDelete = (id) =>
rpc("assignment_delete", { p_token:
getToken(), p_id: id });
export const assignmentsForClass =
(classId) =>
  rpc("assignments_for_class", { p_token:
getToken(), p_class: classId || null });
export const submissionsFor =
(assignmentId) =>
  rpc("submissions_for", { p_token:
getToken(), p_assignment: assignmentId });
export const submissionMark = (id, score,
outOf, comment) =>
  rpc("submission_mark", { p_token:
getToken(), p_id: id, p_score: score,
p_out_of: outOf, p_comment: comment });

// The family side carries the admission
number and PIN each time rather than a
// session, so a household can only ever
reach its own child's work.
export const assignmentsForStudent = (adm,
pin) =>
  rpc("assignments_for_student", { p_adm:
adm, p_pin: pin });

```

```

export const submissionSend = (adm, pin,
assignmentId, answer, photo) =>
  rpc("submission_send", { p_adm: adm,
p_pin: pin, p_assignment: assignmentId,
p_answer: answer
|| null, p_photo: photo || null });

// ----- files attached to work -----
----
export const workFileAdd = (assignmentId,
filename, mime, data) =>
  rpc("work_file_add", { p_token:
getToken(), p_assignment: assignmentId,
p_filename:
filename, p_mime: mime, p_data: data });
export const workFileDelete = (id) =>
  rpc("work_file_delete", { p_token:
getToken(), p_id: id });
export const workFilesList =
(assignmentId) =>
  rpc("work_files_list", { p_token:
getToken(), p_assignment: assignmentId });
export const workFilesForStudent = (adm,
pin, assignmentId) =>
  rpc("work_files_for_student", { p_adm:
adm, p_pin: pin, p_assignment: assignmentId
});
export const submissionFileAdd = (adm, pin,
assignmentId, filename, mime, data) =>
  rpc("submission_file_add", { p_adm: adm,
p_pin: pin, p_assignment: assignmentId,
p_filename:
filename, p_mime: mime, p_data: data });
export const submissionFilesList =

```

```

(submissionId, adm, pin) =>
  rpc("submission_files_list", { p_token:
    adm ? null : getToken(), p_submission:
    submissionId,
                                p_adm: adm
    || null, p_pin: pin || null });
export const storageUsed = () =>
  rpc("storage_used", { p_token: getToken()
  });

// Fetches a file and hands it to the
// browser to save. Only the file actually
// wanted is ever downloaded – listings
// carry names and sizes only.
export async function downloadWorkFile(id,
  adm, pin) {
  const rows = await rpc("work_file_get", {
    p_token: adm ? null : getToken(), p_id:
    id, p_adm: adm || null, p_pin: pin || null,
    });
  const f = Array.isArray(rows) ? rows[0] :
  rows;
  if (!f) throw new Error("That file could
  not be fetched.");

  const bin = atob(f.data);
  const bytes = new Uint8Array(bin.length);
  for (let i = 0; i < bin.length; i++)
    bytes[i] = bin.charCodeAt(i);
  const url = URL.createObjectURL(new
  Blob([bytes], { type: f.mime ||
  "application/octet-stream" }));

  const a = document.createElement("a");

```

```

    a.href = url; a.download = f.filename;
    document.body.appendChild(a); a.click();
    a.remove();
    setTimeout(() =>
URL.revokeObjectURL(url), 30000);
    return f.filename;
}

// Reads a chosen file as base64, refusing
anything too large before it is sent.
export function readFileAsBase64(file,
maxBytes = 4200000) {
    return new Promise((resolve, reject) => {
        if (file.size > maxBytes) {
            return reject(new Error(
                `${file.name} is ${(file.size /
1048576).toFixed(1)} MB. The limit is about
4 MB - ` +
                `families pay for every megabyte
they download.`));
        }
        const r = new FileReader();
        r.onload = () => resolve({ name:
file.name, mime: file.type ||
"application/octet-stream",
                                data:
String(r.result).split(",")[1] });
        r.onerror = () => reject(new
Error("That file could not be read."));
        r.readAsDataURL(file);
    });
}

// ----- passwords and sessions -----

```



```

----
// Anyone signed in may change their own;
the current one is required so a
// phone left unlocked cannot be used to
take the account over.
export const changeOwnPassword = (current,
next) =>
  rpc("staff_change_own_password", {
p_token: getToken(), p_current: current,
p_new: next });
export const sessionsList = ()      =>
  rpc("sessions_list", { p_token:
getToken() });
export const sessionsRevoke = (username) =>
  rpc("sessions_revoke", { p_token:
getToken(), p_username: username });
export const securityRecent = (days)   =>
  rpc("security_recent", { p_token:
getToken(), p_days: days || 30 });

// The same rules the database applies,
checked as the person types so they are
// not told only after pressing the button.
export function passwordProblem(pw,
username) {
  if (!pw || pw.length < 8) {
    return "Use at least 8 characters. A
short phrase you will remember is better
than a short word.";
  }
  const low = pw.toLowerCase();
  if (/^[0-9]+$/.test(low)) return "Digits
alone are guessed quickly. Add some
letters.";
}

```

```

const common =
["password","12345678","qwertyui","password
1","abc12345","11111111",

"teacher1","school123","kenya123","banane12
3","shantrol"];
  if (common.includes(low)) return "That is
one of the first passwords anyone would
try. Choose another.";
  if (username && low ===
String(username).toLowerCase()) {
    return "The password cannot be the same
as the username.";
  }
  return null;
}

// ----- the schools this portal
serves (owner only) -----
export const ownerSchools = () =>
rpc("owner_schools", { p_token: getToken()
});
export const schoolCreate = (s) =>
rpc("school_create", {
  p_token: getToken(), p_code: s.code,
p_name: s.name, p_location: s.location ||
null,
  p_admin_user: s.adminUser, p_admin_name:
s.adminName, p_admin_password:
s.adminPassword,
  p_stage: s.stage || "basic",
});
export const schoolSetActive = (code,
active) =>

```

```

    rpc("school_set_active", { p_token:
getToken(), p_code: code, p_active: active
});

// ----- one link per school -----
-
// A school reached at ?school=sarif
behaves as if it were the only school in
// the world: its own name, its own colours
of speech, no picker. That is what
// a head teacher should be given – a link
to their school, not a list of
// everybody's.
export function schoolFromUrl() {
  try {
    // 1. the subdomain, which is the
address a school should be given:
    //   sarif.gtechschools.co.ke – the
domain itself says whose it is.
    const host =
window.location.hostname.toLowerCase();
    const parts = host.split(".");
    // Ignore hosts that cannot carry a
school subdomain: bare domains, an IP,
    // localhost, and Vercel's own preview
addresses.
    const RESERVED = ["www", "app",
"portal", "admin", "api", "staging",
"preview", "test"];
    const isIp =
/^\\d+\\.\\d+\\.\\d+\\.\\d+$/ .test(host);
    // A two-part ending like .co.ke means
the bare domain already has three
    // parts – gtechschools.co.ke is not a

```

```

school called "gtechschoools". Count
    // the parts the ending itself uses
before deciding there is a subdomain.
    const TWO_PART =
["co.ke", "or.ke", "ac.ke", "go.ke", "ne.ke", "s
c.ke", "me.ke", "mobi.ke",

"co.uk", "org.uk", "ac.uk", "gov.uk", "com.au",
"co.za", "co.tz", "co.ug"];
    const endsTwoPart = TWO_PART.some((t)
=> host.endsWith("." + t) || host === t);
    const minParts = endsTwoPart ? 4 : 3;
    if (!isIp && parts.length >= minParts
&& !host.endsWith("vercel.app") &&
!host.endsWith("pages.dev")) {
        const sub = parts[0].replace(/^[a-z0-
9]/g, "");
        if (sub && !RESERVED.includes(sub))
return sub;
    }

    // 2. ?school=sarif – works on any
address, including vercel.app
    const q = new
URLSearchParams(window.location.search).get
("school");
    if (q) return
q.toLowerCase().replace(/^[a-z0-9]/g, "");

    // 3. a bare hash, in case a link gets
mangled in a message
    const h = (window.location.hash ||
"").replace(/^#/ , "");
    if (h && /^[a-z0-9]+$/.test(h)) return

```

```

h.toLowerCase();
  } catch (e) {}
  return "";
}

// The address to hand a school. Uses a
// subdomain once a real domain is in
// place, and falls back to ?school= on a
// vercel.app address.
export function schoolUrl(code) {
  try {
    const host =
window.location.hostname.toLowerCase();
    const proto = window.location.protocol;
    const RESERVED = ["www", "app",
"portal", "admin", "api", "staging",
"preview", "test"];
    const firstLabel = host.split(".")[0];

    // A wildcard-per-school setup was
    // never actually configured in DNS –
    // Cloudflare's free plan does not add
    // one automatically, and only a
    // handful of subdomains have ever been
    // added by hand. So on the app's
    // own reserved address
    // (portal.cxm.co.ke) – as well as
    // Vercel/localhost
    // – generate a link that needs zero
    // extra DNS: one address, the school
    // named as a query. This is what stops
    // "portal" itself from being
    // mistaken for a school and turned
    // into banane.cxm.co.ke.

```

```

    if (host.endsWith("vercel.app") ||
host.endsWith("pages.dev") || host ===
"localhost" ||
        /\^d+\.\.d+\.\.d+\.\.d+$/\.test(host)
|| RESERVED.includes(firstLabel)) {
        return
`${window.location.origin}${window.location
.pathname}?school=${code}`;
    }
    // strip any existing school subdomain
before adding this one
    const parts = host.split(".");
    const TWO_PART =
["co.ke", "or.ke", "ac.ke", "go.ke", "ne.ke", "s
c.ke", "me.ke", "mobi.ke",

"co.uk", "org.uk", "ac.uk", "gov.uk", "com.au",
"co.za", "co.tz", "co.ug"];
    const endsTwoPart = TWO_PART.some((t)
=> host.endsWith("." + t) || host === t);
    const minParts = endsTwoPart ? 4 : 3;
    const base = parts.length >= minParts ?
parts.slice(1).join(".") : host;
    return `${proto}://${code}.${base}/`;
  } catch (e) {
    return `?school=${code}`;
  }
}

// The school's own details, for the sign-
in screen, before anyone signs in.
export const schoolByCode = async (code) =>
{
  const rows = await rpc("schools_list",

```

```

});
  return (rows || []).find((s) => s.code
=== code) || null;
};

// Anyone signed in may set their own email
and phone. Until now only an
// administrator could, which meant the
staff who most need self-service reset
// – the ones who cannot easily reach the
administrator – could not enable it.
export const setMyContact = (email, phone)
=>
  rpc("staff_set_my_contact", { p_token:
getToken(), p_email: email || null,
p_phone: phone || null });

// ----- the school's crest -----
// Read by school code, because the sign-in
screen needs it before anyone has
// signed in. A crest is on the gate, not a
secret.
export const logoGet = (schoolCode) =>
rpc("logo_get", { p_school_code: schoolCode
});
export const logoSet = (dataUrl) =>
rpc("logo_set", { p_token: getToken(),
p_data: dataUrl || null });

// A crest is a small mark on a page, and
every parent downloads it on every
// visit – so it is shrunk hard before it
is ever sent.
export async function shrinkLogo(file,

```

```
maxEdge = 320) {
  if (file.type === "image/svg+xml") {
    // an svg is already small and scales
    perfectly; send it as it is
    const text = await file.text();
    if (text.length > 200000) throw new
    Error("That drawing is too large. Save it
    as a PNG instead.");
    return "data:image/svg+xml;base64," +
    btoa(unescape(encodeURIComponent(text)));
  }
  const dataUrl = await new Promise((res,
  rej) => {
    const r = new FileReader();
    r.onload = () => res(r.result);
    r.onerror = () => rej(new Error("That
    file could not be read."));
    r.readAsDataURL(file);
  });
  const img = await new Promise((res, rej)
  => {
    const i = new Image();
    i.onload = () => res(i);
    i.onerror = () => rej(new Error("That
    file is not a usable image."));
    i.src = dataUrl;
  });
  const scale = Math.min(1, maxEdge /
  Math.max(img.width, img.height));
  const w = Math.round(img.width * scale),
  h = Math.round(img.height * scale);
  const canvas =
  document.createElement("canvas");
  canvas.width = w; canvas.height = h;
```



```
// transparency matters on a crest, so  
png is kept rather than flattened  
canvas.getContext("2d").drawImage(img, 0,  
0, w, h);  
return canvas.toDataURL("image/png");  
}
```